

dsh 插件化深度解析 —— 一切皆插件是如何运转的

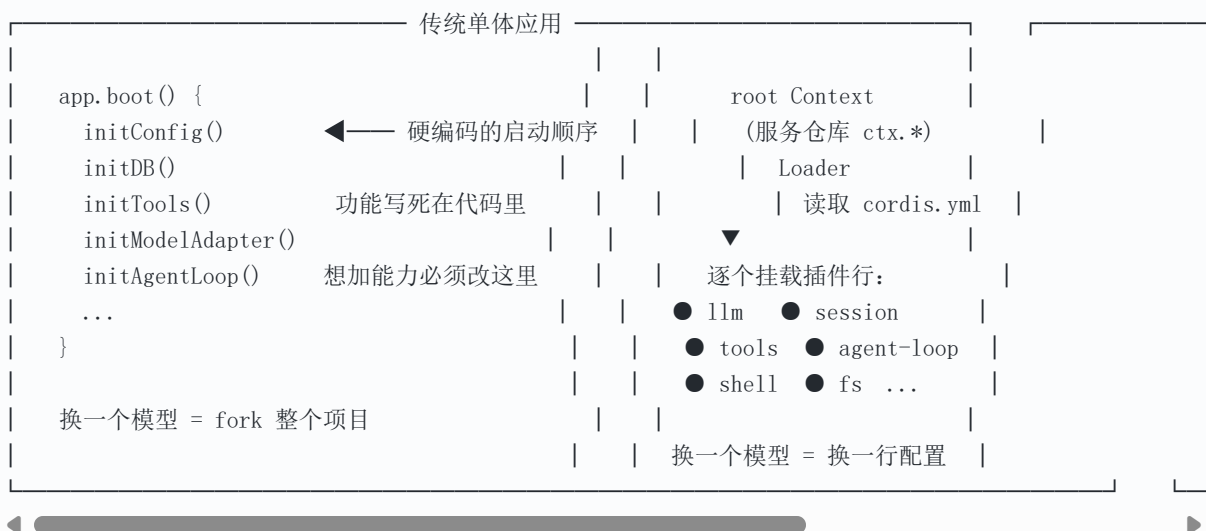
面向 AI 应用开发者的底层原理解析 · 基于 deepseek-harness v0.1.1-rc.2 (b150a55) 源码 · 系列第二篇

本文是「dsh 构建细节与大模型知识」系列第二篇。第一篇见 [dsh 图像管线深度解析](#)。

1. 为什么 dsh 选择「一切皆插件」
2. Cordis 五件事：插件框架的内核
3. 一个插件的真实形态
4. Loader 与 cordis.yml：应用如何被组合
5. 运行时的形状：插件树与 fiber
6. Profiles / Bundles / Layers：可分发、可叠加的插件化
7. 能力缝（capability seam）：一次替换改变整个产品
8. 设计原理：为什么这样设计
9. 热重载（HMR）：改代码不重启
10. 与同类产品的对比
11. 实践建议
12. 术语速查

一、为什么 dsh 选择「一切皆插件」

先建立一个最重要的认知：**dsh 不需要打补丁的「特权核心」**。模型适配器、工具注册表、会话日志、甚至 agent loop 本身，全部都是插件。



`architecture.md` 里把这句话说得很直白:

不需要打补丁的特权核心: 你通过在其它插件旁边挂一个插件来扩展 dsh, 而注册都是 effect, 会在所属插件卸载时回卷。

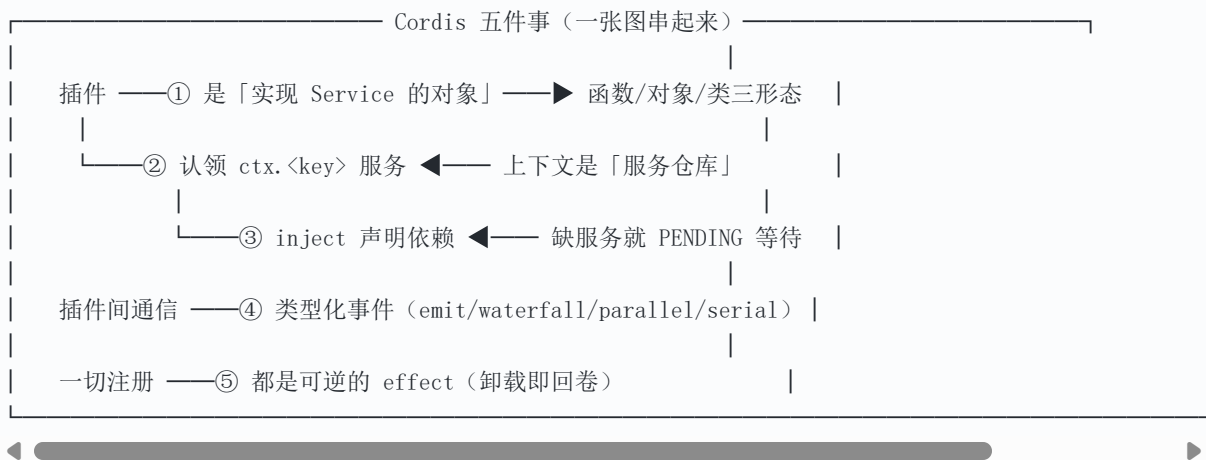
这意味着三件事:

- **可替换性来自配置, 而非分叉**——换一个 LLM 适配器、换一个 shell 执行世界、换一个 subagent 实现, 都只是换一棵插件树里的几行, 不需要 fork 源码。
- **能力有统一生命周期**——任何贡献 (工具、事件监听、服务、适配器) 都走同一条「加载即注册、卸载即撤销」的路径。
- **组合是可声明的**——应用长什么样, 由一份分层叠加的配置决定, 而不是由 `boot()` 里的调用顺序决定。

理解后面所有内容, 只要抓住一句话: **运行中的 dsh 是一棵在 boot 时由有序层组合出来的插件树。**

二、Cordis 五件事: 插件框架的内核

`docs/cordis-primer.md` 把 Cordis (dsh 底层那套被 vendored 的插件框架) 归纳成五件事。这是全文的基石, 也是后面「设计原理」一章的出发点。



2.1 一个插件是「实现了 Service 的对象」

它可以是三种形态之一：

```
import { Service, type Context } from '@deepseek-ai/cordis'

// 1. 函数插件（最常见）：可选 name / inject，必须有 apply(ctx)
export const name = 'hello'
export function apply(ctx: Context) {
  console.log('hello from my first plugin')
}

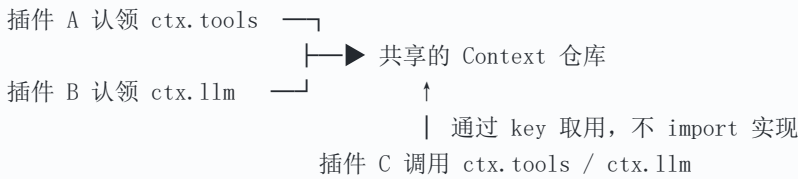
// 2. 对象插件：带 apply 方法的对象
export const objectPlugin = { name: 'object-plugin', apply(ctx: Context) {} }

// 3. 类插件：Service 的子类（需要公开服务时用，见第三章）
export class MyService extends Service {
  constructor(ctx: Context) { super(ctx, 'myTutorialService') }
}
```

在你需要公开服务之前，请一直用函数形态。来源：[docs/cordis-tutorial/01-first-plugin.zh.md](https://docs.cordis-tutorial/01-first-plugin.zh.md)。

2.2 上下文是「服务的仓库」

一个服务从上下文里认领一个稳定的 `ctx.<key>`，比如 `ctx.tools`、`ctx.llm`、`ctx.sessions`；其它插件通过 **key** 而不是 import 某个具体实现来找到它。



2.3 用 inject 声明依赖

一个插件若 `inject: ['timer']`，它会一直等到 `timer` 服务出现才加载。于是**加载顺序由服务依赖表达，而不是手写 boot 时序**。这正是 2.2 那句「通过 key 找服务」的另一面：找不到就等，不报错、不崩溃。

2.4 类型化事件做通信

服务通过 TypeScript 声明合并（declaration merging）声明事件名，再以四种模式分发：

模式	等待?	触发顺序	有返回值?	典型用途
<code>emit</code>	否	按注册顺序观察	否	广播事实
<code>waterfall</code>	否	按注册顺序观察	是	中间件式改写/拦截
<code>parallel</code>	是	全部并发观察	否	并发副作用
<code>serial</code>	是	按注册顺序	是	有序决策

`ctx.waterfall` 是 around 中间件：监听器拿到 `(...args, next)`，调用 `next()` 把（可能被包装过的）结果往下传；**不调用 `next()` 就是短路**。单决策事件里短路是设计本身（策略监听器直接 own 决策）；只做标注的监听器必须 `next()`。`architecture.md` 与 `packages/AGENTS.md` 都强调：**`waterfall` 监听器必须调用 `next()`**，否则会截断整条链。

2.5 注册是可逆的 effect

工具 schema、prompt 段落、适配器、provider、事件监听，全部通过 `ctx.effect()` 或 `ctx.on()` 安装，于是**热重载和拆除都能可预测地回卷**。你很少需要亲手写 `ctx.effect()`，因为内置注册 API 本身已经是 effect：

`ctx.on(event, listener)` 卸载时移除；`ctx.plugin(child)` 随父插件一起 dispose；

`ctx.tools.register(...)` 这类 harness 注册表也会把返回的 disposer 挂在调用插件上自动撤销。

三、一个插件的真实形态

3.1 函数插件（贡献行为，不公开服务）

```
import type { Context } from '@deepseek-ai/cordis'

export const name = 'lifecycle-demo'

export function apply(ctx: Context) {
  // 任何 Cordis 没管的资源（定时器/连接/watcher）都包进 effect，并返回释放函数
  ctx.effect(() => {
    const timer = setInterval(() => console.log('tick'), 200)
    return () => { // 卸载时运行
      clearInterval(timer)
      console.log('cleaned up')
    }
  })
}
```

要点（<docs/cordis-tutorial/02-lifecycle-and-effects.zh.md>）：

- effect 主体在加载期运行，返回的 disposer 在卸载期运行；
- 对于生命周期与插件一致的资源，你绝不需要自己调 disposer；
- `ctx.plugin(child)` 会返回一个 **fiber**（已加载插件实例的运行时句柄），`fiber.dispose()` 会等它的全部清理（含异步）完成后，再递归卸载它挂的所有子插件。

3.2 服务插件（公开一个 `ctx.<key>`）

```
import { Service, type Context } from '@deepseek-ai/cordis'

export class MyService extends Service {
  constructor(ctx: Context) { super(ctx, 'myService') }
  doWork() { /* ... */ }
}

// 其它插件写 inject: ['myService']，然后通过 ctx.myService.doWork() 调用
```

类插件把能力挂上 `ctx` 的某个 key，让别的插件按 key 依赖它——这就是 2.2/2.3 的组合闭环。

3.3 双形态陷阱（来自 dsh-hub 的踩坑记录）

dsh-hub/AGENTS.md 特别点出：服务包默认导出它的类；函数插件用命名导出 `name` / `inject` / `Config` / `apply`，且不要有默认导出。把两者混用，Loader 会丢掉函数插件的命名空间。注册都是 effect：`ctx.effect()`、`ctx.on()`（含 waterfall 监听）——卸载插件 fiber 会把它们全部反注册。

四、Loader 与 cordis.yml：应用如何被组合

Cordis 不是让你在代码里 `import` 插件并调用，而是让你描述一份插件清单，由 Loader 去挂载。

```
# cordis.yml — 一组 Cordis 配置项的列表
- id: logger
  name: '@deepseek-ai/cordis-plugin-logger-console'
- id: hello
  name: './hello.ts'           # 相对路径或 npm 包名都行
- id: heavy
  name: './heavy.ts'
  disabled: true               # 保留配置项但不挂载；改回 false 即重新加载
```

docs/cordis-tutorial/01-first-plugin.zh.md 解释了几件事：

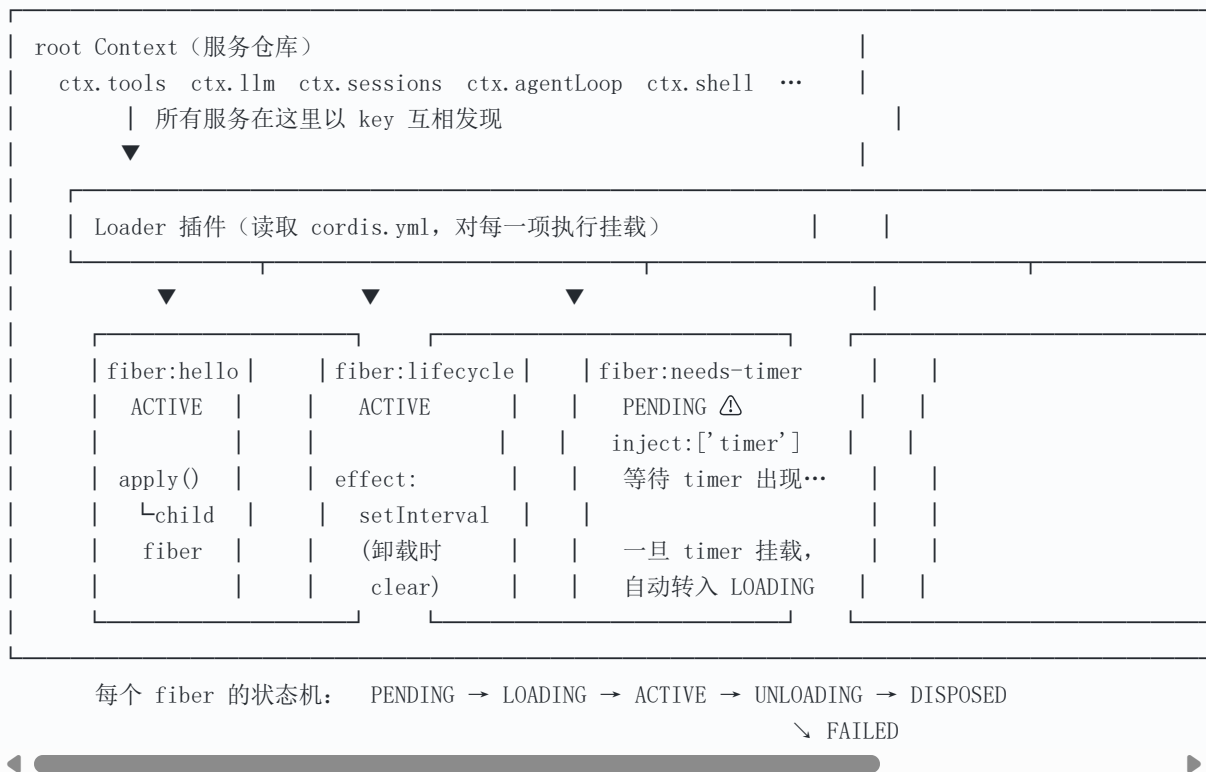
- `name` 是模块指定符（相对路径或包名），Loader 挂载每一项；
- 各项并发启动，所以它们在列表里的位置不保证加载先后——顺序由服务依赖（`inject`）决定；
- 一个配置项不只有 `name` / `config`，还接受 `id`（稳定标识，让 Loader 能区分「改现有项」与「先删再加」）和 `disabled`。

docs/cordis-primer.md 的「Loader Configuration」补充了配置注入细节：loader 用 `@deepseek-ai/cordis-plugin-include` 把 `!!js` 解析成表达式节点；它会插值在配置项声明注入激活后、`config` 字段上（对该插件上下文 `ctx.serviceName`）以及每次挂载决策时、`disabled` 字段上（对 loader 上下文）进行。其它元数据保持字面量。当环境要挑选插件时，用 overlay（叠加层）而不是条件分支。

`cordis.yml` 与 `cordis.patch.yml` 是同一种行列表的两种用途：前者是独立应用的 loader 入口清单（教程里那种）；后者作为「层」被叠加进一个 profile，按 `id` 插入新行或整体替换某一行的 `config`。本文第六章的 bundle patch 用的就是后者。

五、运行时的形状：插件树与 fiber

把 2~4 拼起来，运行时是这样一棵东西：



每个已加载的插件实例都有一个 **fiber**，在以下状态间转换（[docs/cordis-tutorial/02-lifecycle-and-effects.zh.md](#)）：

- **PENDING**：已声明，但所需服务尚不可用——这是「我的插件怎么没输出」最常见的答案；
- **LOADING / ACTIVE**：`apply` 正在跑 / 已跑完；
- **FAILED**：`apply` 或配置校验抛异常；
- **UNLOADING / DISPOSED**：disposer 正在跑 / 全部拆完。

依赖缺失是静默的，不是错误。 如果 `inject` 指定了没人提供的服务，插件就一直 **PENDING**，不输出任何东西。诊断方法（[docs/cordis-tutorial/06-composition-and-hmr.zh.md](#)）是枚举注册表、打印处于 **PENDING** 的 fiber：

```
import { FiberState, type Context } from '@deepseek-ai/cordis'
export function apply(ctx: Context) {
  setTimeout(() => {
    for (const runtime of ctx.registry.values())
      for (const fiber of runtime.fibers)
        if (fiber.state === FiberState.PENDING)
          console.log(`${fiber.name} is PENDING - a required service is missing`)
  }, 500)
}
```

六、Profiles / Bundles / Layers: 可分发、可叠加的插件化

单个 `cordis.yml` 只是玩具。真实 dsh 把「插件清单」升级成三层概念（`docs/architecture.md` 的 Profiles and bundles）：

运行中的 dsh 是一棵在 boot 时由**有序层（ordered layers）**组合出来的插件树。

6.1 三个概念

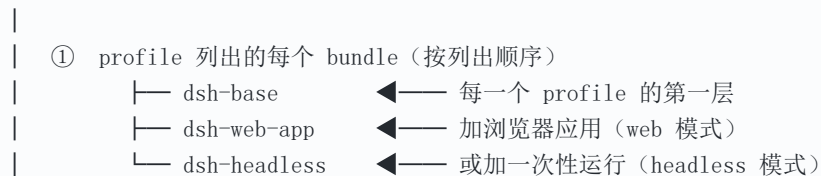
- **Profile（配置方案）**：存放在 Harness home 里的一份**命名组合**。它列出要堆叠的 bundles、收纳它额外安装的 out-of-tree 插件、并持用户自己的 `cordis.patch.yml`。`web` 和 `headless` 作为模板随产物发布。
- **Bundle（插件包）**：Cordis 配置行 + 它们挂载的代码的**发行格式**。关键性质——它插入的内容仍能被它**上面的层** patch，所以任何 bundle 都不是黑盒。
- **Layer（层）**：叠加的顺序单位。每个 bundle、用户的 patch、home 级 patch、`--patch` overlay 各自是一层。

三者怎么声明？各自在 `package.json` 里用 `dsh` 字段自报家门：`dsh.profile` 列出一个 profile 包含的 bundles；`dsh.bundle` 指向该 bundle 的 patch 文件。

6.2 层是怎么叠加的

层按顺序作用在一个**空的入口列表**上，后写的层可以覆盖先写的层（按 `id` 整体替换）：

空入口列表



▼
| ② profile 自己的 `cordis.patch.yml`

▼
| ③ home 级 `cordis.patch.yml`

▼
| ④ `--patch` overlay（按环境选择）

▼
最终插件树

└─ 同一 id 被多层写入时：以「最后写入者」胜（last write wins per row）

两条规则（`docs/architecture.md` + `packages/bundle/base/cordis.patch.yml` 注释）：

- **patch 按 id 定位某一行，替换的是该行「整个 config」**，而不是 merge 进去——所以「同一行在不同模式下值不同」不该塞进 base，而该属于各模式 bundle；
- **同一行多个层都写，以最后写入者胜 (last write wins per row)**。

dsh-base 是每一个 profile 的第一层（模型适配器、工具、持久化、sandbox、审批策略、设置、凭据、遥测）；**dsh-web-app** 加上浏览器应用；**dsh-headless** 加上「一次性运行、完全不带服务器」的模式。想看你这台机器真正 boot 出来的树：

```
dsh --profile web --dump-config
```

它打印的每一行，你都能用自己的 patch 替换掉。

6.3 一份真实的 bundle patch

`packages/bundle/base/cordis.patch.yml` 长这样（节选）：

```
# dsh-base: 每个 profile 共享核心，作为「对空 profile 根的一次 insert」施加。
# 后续 bundle patch 与用户 profile 的 cordis.patch.yml 按 id 定位这些行，
# 同一行以最后写入者胜。
- insert:
  - id: timer
    name: '@deepseek-ai/cordis-plugin-timer'
  - id: hmr
    name: '@deepseek-ai/cordis-plugin-hmr'
    config:
      root: ['.']
  - id: llm
    name: '@deepseek-ai/dsh-llm'
  - id: session
    name: '@deepseek-ai/dsh-session'
  - id: typert
    name: '@deepseek-ai/dsh-typert-registry'
# ..... web / headless 等模式 bundle 再在这些 id 上 restate 自己的完整 config
```

注意 **id** 是全局的（跨整个 profile）。dsh-hub 的 **AGENTS.md** 建议：用能力风格的前缀（如 **my-bash-shell**）给每行起稳定的 id，方便后续 patch 精准命中。

6.4 用户怎么装一个插件（面向 dsh-hub）

在 dsh-hub 这套指针仓库里，装一个第三方插件走的是 **dsh plugin add**（**dsh-hub/AGENTS.md** 的插件规约）：

```
dsh plugin add github:<owner>/<repo>      # GitHub: 需 prepare 从源码构建 lib/
dsh plugin add <npm-package>                # npm: 要求发布时已构建好 lib/
dsh plugin add ./local/path                 # 本地开发
```

一个能被 `dsh plugin add` 装上的仓库必须满足（非谈判项）：

- `package.json` : `type: "module"`、`main: "lib/index.js"`、`exports["."]`、声明 `dsh.bundle.patch: ./cordis.patch.yml`；`@deepseek-ai/cordis` 同时进 `peerDependencies` 与 `devDependencies`（否则会出现重复 Cordis 实例、ctx 坏掉）；`@deepseek-ai/schemastery` 进 `dependencies`；scripts 含 `build` / `prepare` / `typecheck` / `lint`；
- `cordis.patch.yml` : 一个 `- insert:` 列表，每行有全局唯一的 `id`、`name`（从 profile 的 `node_modules` 解析）、`config`；
- `src/index.ts` : 函数插件命名导出 `name` / `inject` / `Config` / `apply` 且无默认导出；服务包默认导出类。

验证是强制的：建一个临时 profile，对 github / npm / local 三条路径各跑一次 `dsh plugin --profile <tmp> add`，确认 `lib/` 经 `prepare` 构建、无 `dsh plugin` 警告、`dsh --dump-config` 显示该层已激活。

6.5 一个真实的「用户叠加」例子

假设 base 已经用 `id: llm` 插入了模型适配器。用户的 profile `cordis.patch.yml` 可以在它之后叠加，按同一个 `id` 改写配置，或新增 base 里没有的行：

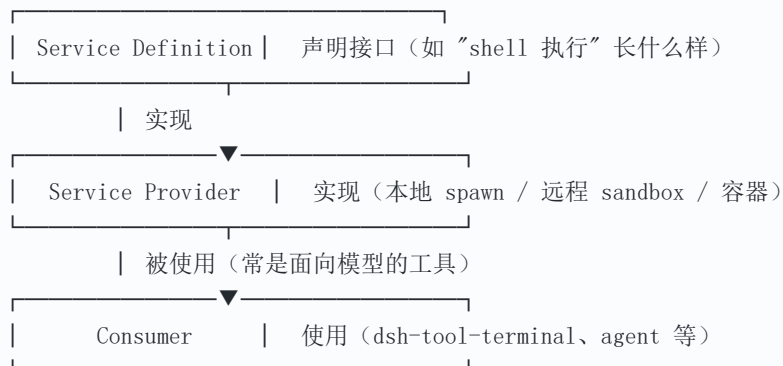
```
# 用户的 profile cordis.patch.yml（作用于 base 之后）
- insert:
  - id: llm
    name: '@deepseek-ai/dsh-llm'
    config:
      defaultProvider: deepseek-reasoner  # 整体替换 base 里 llm 的 config
  - id: my-bash-sandbox
    name: 'dsh-sandbox-my'                # 新增一行，base 里没有
    config:
      network: isolated
```

因为「同一 `id` 以最后写入者胜」，这里的 `llm` 配置会盖掉 base 的那份；`my-bash-sandbox` 则是全新插入的一行。这就是 bundle 为什么说「插入的内容仍可被上层 patch」——它从设计上就不是黑盒。

七、能力缝（capability seam）：一次替换改变整个产品

`docs/architecture.md` 把 dsh 可替换性的真正引擎叫 **seam（能力缝）**：

一个 seam 是一个可替换的能力，有三个角色：声明接口的 **Service Definition**、实现它的 **Service Provider**、使用它的 **Consumer**（通常是面向模型的工具）。一个包可以合演多角，但**只有一角是构不成 seam 的**；加一个能力意味着三个角色都设计出来。



三者构成一个完整、可替换的能力单元；缺任何一角都不叫 seam。

换一个 Provider（例如指向远程 sandbox）——► 整片行为跟着变。

缝之所以强大，是因为「换一个 provider 就换了整个产品」：

- 文件系统与子进程的 provider 共享**同一个执行世界**，所以把这两者指向一个远程 sandbox，Bash、PTY、LSP 会**一起**被搬过去，无需各自分叉 provider；
- subagent 的 provider 在同一个接口后面差异极大——从「一个全新的子 agent」到「另一个产品里被委派的一轮」，可以无缝换。

`architecture.md` 还列了一张「新行为去哪」的表（节选），几乎覆盖了所有扩展点：

目标	机制
加一个模型 provider	在 <code>ctx.llm</code> 上注册它的适配器
加一个面向模型的能力	在 <code>ctx.tools</code> 注册；它的 schema 会并入 prompt 组装
加文件系统访问或策略	注册一个 <code>ctx.fs</code> provider，或监听 <code>fs/*</code> 事件
让某会话用不同的能力集	组合一个 agent preset；那一行服务需要 <code>isolate</code> 一个 realm
加 shell 执行	注册一个 <code>ctx.shell</code> 后端；本地实现经 <code>ctx.subprocess</code> 拉起
加持久终端	注册一个 <code>ctx.terminals</code> 后端 + <code>dsh-tool-terminal</code>
加人类命令	注册到 <code>ctx.commands</code> ；它不走模型轮次直接分发
加后台工作	注册到 <code>ctx.jobs</code> ； <code>job_*</code> 工具收集或停止它
拦截一次请求/工具/轮次	用它的 <code>agent/*</code> 或 <code>tools/*</code> 事件； <code>agent/turn-stopping</code> 能停下整轮
加面向模型的上下文	调 <code>agent.inject()</code> ；它落到下一个被接纳的请求里
fork 一个活跃会话	<code>ctx.sessions.fork(source, boundary?, childSessionId?)</code>
把注册限定在某个 agent	用那个 agent 的 <code>agent.ctx</code>

关于上表中的 `isolate`：它给一个组提供某项服务名称的**独立实例**，于是两个组可以各自看到配置不同的 provider，互不影响（[docs/cordis-tutorial/06-composition-and-hmr.zh.md](#)）。agent preset 用它是为了让「某个会话」拥有不同于全局的能力集，而不污染其它会话。

八、设计原理：为什么这样设计

前面讲的都是「是什么」，这一章讲「为什么」——把五件事和三层概念背后的**底层原理**摊开。理解了这些原理，你就能预判 dsh 在任何新场景下会怎么行为。

8.1 原理一：无特权核心 → 可替换性来自配置，而非分叉

传统框架把「核心」写死，扩展要靠继承、钩子点或 fork。dsh 反过来：**任何核心能力本身也是插件**，于是「扩展」和「替换核心」是同一个动作——挂一个插件 / 换一行配置。

- **为什么成立**：因为所有贡献都通过同一个 `ctx` 注册，不存在「框架内部」和「用户代码」的二分。
- **没有它会怎样**：换一个模型或执行后端就要 fork 整个 harness，每追一次上游就重合一次冲突。

8.2 原理二：依赖驱动加载 → 去中心化耦合，没有中央编排器

加载顺序不靠一个 `boot()` 手动排，而靠 `inject` 声明的服务依赖。缺依赖的插件安静地停在 `PENDING`，等依赖出现再加载。

- **为什么成立**：插件只声明「我需要 `timer`」，不关心 `timer` 是谁、何时挂上；顺序由一张依赖图自然涌现。
- **没有它会怎样**：`boot()` 里 `initA(); initB()` 一旦依赖反过来，就要手动改调用顺序，且容易因循环依赖炸锅。

8.3 原理三：注册即 effect → 可预测的生命周期与可逆热更新

所有注册（工具、事件、服务、子插件）都挂在「插件 fiber」上，卸载时统一回卷。**注册与卸载对称**是整张设计网的支点。

- **为什么成立**：`ctx.effect()` / `ctx.on()` 把「资源获取」和「释放」写成一个闭包的两段，框架保证加载跑前段、卸载跑后段。
- **没有它会怎样**：热重载、卸载、出错恢复都会泄漏资源（定时器、连接、监听没清掉），或需要每个插件手写脆弱的清理逻辑。

8.4 原理四：分层 patch 按 id、last-write-wins → 开放可覆盖，无需 fork

bundle 不是黑盒：它插入的行带着全局 `id`，上层（profile patch、home patch、overlay）能用同一个 `id` 整体替换其 `config`，也能插入新行。

- **为什么成立**：配置即数据，按 stable `id` 寻址；叠加是确定性的（顺序固定、末写胜），所以结果可复现、可审计。
- **没有它会怎样**：要么每个项目 fork 一份 bundle 改，要么只能用 `if/else` 在配置里分环境——分叉会漂移，分支会腐化。

8.5 原理五：类型化事件 + 声明合并 → 可演进的通信，编译期可检查

事件不是字符串魔法，而是 TypeScript 里通过声明合并定义的「带类型的契约」，`@mode` 标注它的分发模式。

- **为什么成立**：监听器签名由事件类型约束，`emit` / `waterfall` / `parallel` / `serial` 的语义写进类型，重构时编译器帮你查调用点。
- **没有它会怎样**：事件名拼错只剩运行时报错；`waterfall` 忘记 `next()` 这种错误无法静态发现。

8.6 原理六：能力缝 → 子系统级替换

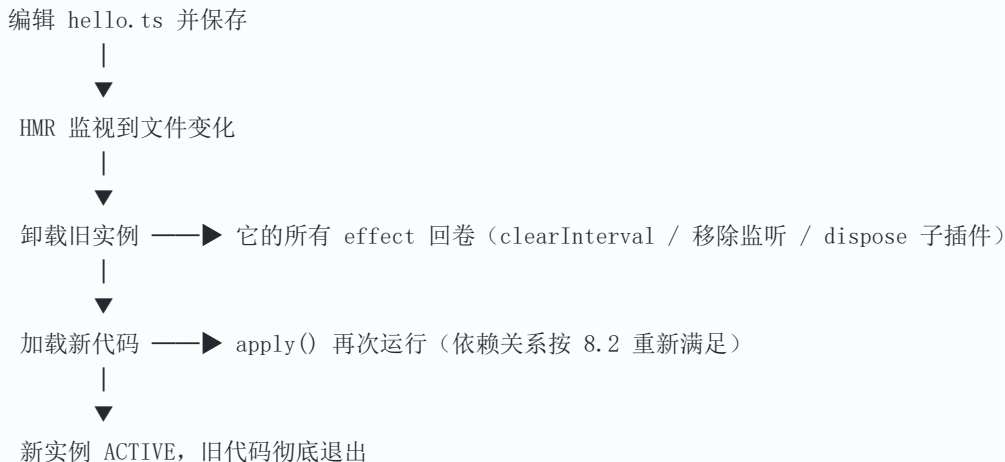
把「一个能力」定义为 Definition / Provider / Consumer 三角色的完整单元，使得**换 Provider 等于换整片行为**，而 Consumer（工具/UI）无需改动。

- **为什么成立**：Consumer 只依赖 Definition 的接口，不依赖 Provider 的实现；所以同一接口后的实现可以彻底不同（本地 vs 远程 sandbox）。
- **没有它会怎样**：能力以「散落的 if」嵌在 loop 里，换后端要改核心代码，且 Bash/PTY/LSP 这类共享执行世界的组件无法一起迁移。

六条原理里，真正统摄全局的是 8.3（effect 可逆）和 8.2（依赖驱动）：它们一起让「声明式组合 + 可预测卸载」成为可能，其余四条（无特权核心、分层 patch、类型化事件、能力缝）都是在这个底座上长出来的。

九、热重载（HMR）：改代码不重启

`@deepseek-ai/cordis-plugin-hmr` 监视文件，保存时执行「先卸载、再加载」来替换正在跑的插件（`docs/cordis-tutorial/06-composition-and-hmr.zh.md`）。



编辑 `hello.ts` 保存后的实际输出：

```
hello from my first plugin
hmr reload plugin at hello.ts
hello from my EDITED plugin
```

这件事成立的前提，正是前文两点：**注册是可逆的 effect**（卸载能干净拆掉，见 8.3），以及**加载是依赖驱动的**（重新加载后依赖关系重新满足，见 8.2）。

`id` 在这里也很关键：不带 `id` 的配置项每次被读取都会获得一个新生成的 `id`，于是只要配置文件有任何编辑——哪怕那一行文本没变——它也会被视为「先删再加」并重新挂载。这就是为什么 `cordis` 配置项要显式带 `id`。

编辑 `cordis.yml` 本身也会触发更新：loader 按 `id` 比较配置项，只挂载/卸载/重配置发生变化的部分。

十、与同类产品的对比

下面把 `dsh` 的插件化，与三款同类智能体产品的扩展机制放在一起对照。**`dsh` 部分基于 `v0.1.1-rc.2` 源码；`opencode` / `Claude Code` / `Codex` 部分基于其公开文档与设计，属于概念层对照，未逐行研读其源码。**

维度	dsh / Cordis	opencode	Claude Code	Codex
内核哲学	一切皆插件 （模型适配器、工具、loop 都是插件）	插件是 npm 包，框架加载其 <code>setup</code> ；核心不是插件	无「一切皆插件」内核；核心是固定 agent loop	以配置 + markdown agent 定义为主，无通用用户插件框架
扩展机制	<code>ctx</code> 服务 + 类型化事件 + effect 注册	<code>Plugin</code> 对象导出 <code>setup(ctx)</code> ，扩展命令/provider 等	plugins (slash 命令 + hooks) + MCP servers + skills	自定义 agent、审批策略、命令行配置
加载顺序	依赖驱动： <code>inject</code> 缺服务 → PENDING	插件 <code>setup</code> 顺序由配置/依赖决定	hooks/插件按配置启用，无强依赖图	启动时按配置装配
配置叠加	分层 patch，按 id last-write-wins	配置合并 (merge)，无按 id 的分层覆盖	settings / hooks / CLAUDE.md 多层，覆盖规则各异	配置项 + 环境变量
核心可替换性	换一个 Provider/适配器 = 换整片行为 (seam)	有限：主要换 provider / 加命令	通过 MCP 接入外部能力；核心 loop 不可换	切内置 provider / 模型，loop 固定
生命周期 / 热	effect 可逆，HMR 天然成立	取决于插件自身实现	基本要重启	基本要重启

更新				
通信模型	类型化事件 (emit/waterfall/parallel/serial) , 声明合并、编译期检查	插件内部事件/ 回调	hooks 事件 (PreToolUse/PostToolUse) + MCP 协议	内部事件, 对用户不可扩

10.1 各自的取舍

- **dsh** 把「可替换性」推到极致：因为一切皆插件、注册皆 effect，所以它用**配置叠加**而非 fork 来定制，用**依赖驱动**而非中央编排来解耦。代价是心智模型更重（你要理解 `ctx`、fiber、layer 这几层），以及配置叠加的确定性要靠 `id` 纪律来维持。
- **opencode** 的插件模型更轻：一个 `Plugin` 对象 + `setup`，上手快，适合「加个命令 / 接个 provider」。它不试图把所有核心都变成插件，因此中央 loop 仍是固定的，换核心能力不如 dsh 灵活。
- **Claude Code** 走的是「hooks + MCP + skills」路线：核心 agent loop 不可替换，但你可以通过 MCP 把任意外部系统接成工具，通过 hooks 在工具调用前后插入逻辑。它的扩展点是「事件拦截 + 外部能力」，而不是「替换内核组件」。
- **Codex** 最克制：几乎没有用户级插件框架，能力切换主要靠内置 provider 与配置。它的设计重心是「可靠地跑一个 agent」，而不是「让用户重组 agent」。

一句话定位：如果你要的是「换一个组件就换一片美的行为、且定制不漂移」，dsh 的 Cordis 分层插件树是为这个场景生的；如果你要的是「快速加个命令 / 接个外部工具」，opencode 的 `Plugin` 或 Claude Code 的 MCP 更轻。两者不是高下之分，而是把「可替换性」放在了不同层级。

十一、实践建议

给插件开发者：

- 默认用**函数形态**；只有要公开 `ctx.<key>` 时才上 Service 子类。服务包默认导出类、函数插件命名导出 `name/inject/Config/apply` 且无默认导出——两者别混。
- 任何 Cordis 没管的资源（定时器、连接、watcher）都包进 `ctx.effect()` 并返回 disposer；内置注册 API 本身已是 effect，别重复造。
- 部署会变的选择放进可校验的 `Config` 字段（走 cordis.yml 改），**别在插件里硬编码 tunables**（`packages/AGENTS.md` 的硬规矩）。
- 配置项务必带稳定 `id`；否则任何编辑都会触发「删了重加」。

给架构师：

- 想做**可分发**的插件化，用 bundle（配置行 + 代码）而不是把逻辑写进某个 profile；bundle 插入的行仍可被上层 patch，所以它是开放的而不是黑盒。

- 想做**可替换**的能力，用 seam 的三角色（Definition / Provider / Consumer）来设计，而不是在 loop 里加 if。换一个 provider 就能换掉整片行为（原理 8.6）。
- 跨 bundle 共享配置时记住：patch 按 id **整体替换** config，不是 merge；模式相关的值交给各模式 bundle 各自 restate（原理 8.4）。

给排查者：

- 插件「没输出、不报错」——先查它的 fiber 是不是 **PENDING**（缺某个 **inject** 的服务，原理 8.2）。
- 改了插件代码想立刻生效——确认 HMR 插件在跑，且你的包带 **id**（原理 8.3）。
- 想知道最终 boot 出什么——**dsh --profile <name> --dump-config**，它打印的每一行都能被你自己的 patch 替换（原理 8.4）。

十二、术语速查

术语	含义
插件 (plugin)	向共享 Context 贡献服务/事件/可逆 effect 的对象；函数、对象、Service 子类三种形态
服务 (service)	认领一个稳定 <code>ctx.<key></code> 的能力单元；其它插件按 key 而非 import 找它
上下文 (Context)	服务仓库； <code>ctx.tools</code> / <code>ctx.llm</code> / <code>ctx.sessions</code> 等都在其上
<code>inject</code>	插件声明所需服务；缺失时插件进入 PENDING，不报错
effect	通过 <code>ctx.effect()</code> / <code>ctx.on()</code> 安装的注册；插件卸载时自动回卷
fiber	一个已加载插件实例的运行时句柄；有 PENDING→ACTIVE→...→DISPOSED 状态机
Profile	Harness home 里的命名组合：列 bundles、收纳 out-of-tree 插件、持用户 patch
Bundle	Cordis 配置行 + 挂载代码的发行格式；插入内容仍可被上层 patch
Layer	叠加顺序单位；顺序为 bundle→profile patch→home patch→ <code>--patch</code> overlay
patch (按 id)	定位某一行，整体替换其 <code>config</code> 或插入新行；同 id 以最后写入者胜
seam (能力缝)	可替换能力，含 Definition/Provider/Consumer 三角色，缺一不构成 seam
waterfall	around 中间件式事件；监听器须调 <code>next()</code> 否则短路
HMR	热模块替换：保存文件时卸载+加载该插件，因 effect 可逆而成立

基于 deepseek-harness v0.1.1-rc.2 (b150a55) 源码 ([docs/cordis-primer.md](#) 、 [docs/architecture.md](#) 、 [docs/cordis-tutorial/*](#) 、 [packages/bundle/base/cordis.patch.yml](#) 、 [packages/AGENTS.md](#)) 与 [dsh-hub/AGENTS.md](#) 撰写 · 2026-08-26 · dsh-hub 项目 · 系列第三篇预告: agent loop 的内部构造